

Team 6: Browser Extension for Tracking Pixels (monoPix-scout)

Mario Belmonte (mbelmont), Adrian Muñoz (adrianmu)

Introduction

The focus of this project is to build from scratch a fully functional Google Chrome browser extension that allows for the detection and optional blocking of tracking pixels found on any site the user navigates. It's as our README.md file reads on the GitHub repository:

“An image/vision-analysis tool for detecting and visualising pixel-level changes across video frames using a ‘MonoPix’-inspired model. Discovers and reveals hidden imagery and tracking assets loaded on the webpages you visit. See what sites are pulling in the background — even when content is invisible or masked by CSS. Developed by Qrytics & TechnicalDree.”

Methodology

Through the use of GitHub, we managed all our files, maintained edit details, and focused on naming conventions, all for the sake of further progress. With an organized, descriptive project, it's easier for others to pick up where our work fell short and continue to innovate upon our scripts. Additionally, we chose to go through and inform the repo users how to get set up and use our extension, what the project structure ultimately looks like, our roadmap of future endeavors, and an invitation to collaborate.

When creating our browser extension's UI, we focused heavily on its ease of use within the Google Chrome environment. Since its creation was through multiple files' worth of code scripts, we made sure every detail was intentional and well thought out prior to incorporating its functionality.

- Our simple iconography represents a sort of viral theme, simple to remember whilst standing out.
- The drop-down for selecting the site mode avoids unnecessary idle clutter.
- Greying out the refresh button makes it visually separate for the user and signals it has a function.
- Forcing to the background small greyed out subtext to provide small updates on the app's functioning.
- Coloring each danger level grey, yellow, and red to signal increasingly worrying threat detection.

At the start of this project, we wanted to focus on tracking pixels, what they were, how to detect them, and hopefully block their presence. However, as the project progressed and we understood its simplistic nature, we chose to pivot and extend our detection practices to potential threats beyond tracking pixel,s such as beacon and fetch calls. Here's our current understanding of each “threat”:

- **Tracking Pixel:** A small image (commonly 1x1 to minimize detection) that gets requested onto the site page from an external URL. The timing of the request and sometimes cookies containing metadata are leaked.
- **Beacon:** A simple API request that sends your basic information to an external URL. It allows for the signaling of mundane information like scrolling and screen time on certain sites. Due to the function's simplicity, site creators often riddle their pages with beacon calls to collect various information regarding their users' tendencies.
- **Fetch:** Similar to a beacon call, a fetch request is also an integrated API command often used by site creators. It requests HTML/JS/CSS scripts from a separate URL to then run on your own page.

For the beacon and fetch calls, it doesn't always mean it could be intentionally malicious, but having this be detected at all should serve as some form of warning that background requests are being performed. To better understand these functions, we made a demo site consisting of a large image to serve as the control test, one 1x1 pixel image to

simulate a tracking pixel, a beacon-like call, and a fetch-like call. Even though the site was heavily rudimentary, it managed to prove to us that our browser extension was able to detect and correctly classify each threat as was intended. The detection method for each threat will be discussed in detail below.

Figure 1

```
// Scoring function to determine danger level
✓ function score(sig) {
  let s = 0;
  if (sig.size && sig.size.w * sig.size.h <= 16) s += 2;
  if (sig.thirdParty) s += 2;
  if (/beacon|fetch|xhr/i.test(sig.reason)) s += 2;
  if (/(\b(pixel|track|open|view|beacon|analytics)\b)/i.test(sig.url)) s += 1;
  return s >= 4 ? "high" : (s >= 2 ? "medium" : "low");
}
```

score(sig) function: written under file background.js

Above, we've attached a section of our code from our background.js file that is directly responsible for the scoring criteria of the danger level assigned to each "threat." For context, the higher this value 's' is, the higher their respective danger level will be once past the thresholds of 2 for medium and 4 for high. Each if-statement holds its own rule as to what should be considered grounds for adding to their threat score 's.' Below, we'll explain each if-statement in order from top to bottom as seen in Figure 1:

1. A pixel width and height amounting to an image area of less than 16 is a common characteristic only found in tracking pixels. This line serves the sole purpose of detecting those tracking pixels.
2. If the URL involved in the function call or image sourcing is detected to be pertaining to a third-party source, that would be considered suspicious.
3. This checks to see whether the function call is using the background API beacon or fetch requests.
4. The keywords found in the associated URLs pertaining to certain calls and image sourcing can, at times, be suspiciously noteworthy. This line reads through the URLs to ascertain whether any of the common keywords are used, such as pixel, track, open, view, beacon, and analytics.

The method of detection for these beacon and fetch calls is misleadingly straightforward. Our first attempt at detecting these calls was to use a basic monkey patch. The original functions are nabbed and tagged onto our custom function wrappers that send a signal to another one of our scripts that's listening and now knows an instance has been called. Unfortunately, our initial version of this hooking script method failed; it clearly couldn't catch any semblance of the calls.

Stumped and lacking ideas, an LLM proposed that perhaps the problem was that our script was stuck running in the 'content script world' instead of the 'page world.' This idea stemmed from how Google Chrome handles the script behaviors of its extensions; it deliberately isolates content scripts from the page, meaning the page only ever sees functions that exist within its own JavaScript world, known as the 'page world.' With this in mind, we adapted our script into an injection that forces itself onto this 'page world' and successfully managed to perform the hooking monkey patch method.

Blocking the tracking pixels was simple to implement. The idea was that we'd replace the tracking pixels (diminutive images) with exact equal-size canvases to avoid the misarrangement of the site's framing structure. Separate from this, we have the integrated Google Chrome extension rules that can be automatically set to block

certain requests. Through this, within the file `compiled_rules.json`, we set it to block any requests using URLs that contain the keywords “beacon” and “pixel.”

Result

[Tracking Pixel Extension Demo](#)

We also evaluated monoPix-scout’s functionality across 6 websites, as shown in Figures 2 and 3 (5 public sites + our local demo page), where it detected 277 tracking pixels total, averaging 46.2 pixels per site. More importantly, 213 pixels (76.9%) were classified as high risk and third-party, meaning that the majority of detected pixels came from external domains and were scored as more suspicious. Overall, the extension successfully detects tracking pixels across real sites and characterizes them by risk level and third-party presence, with some sites containing significantly more than others.

Figure 2

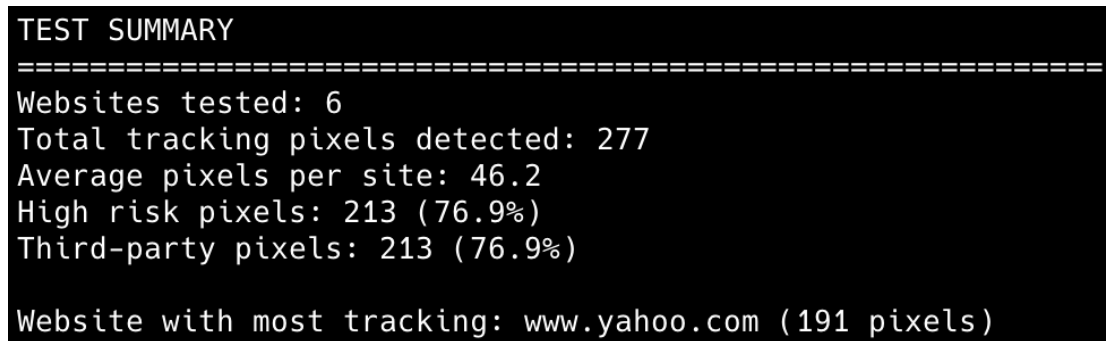
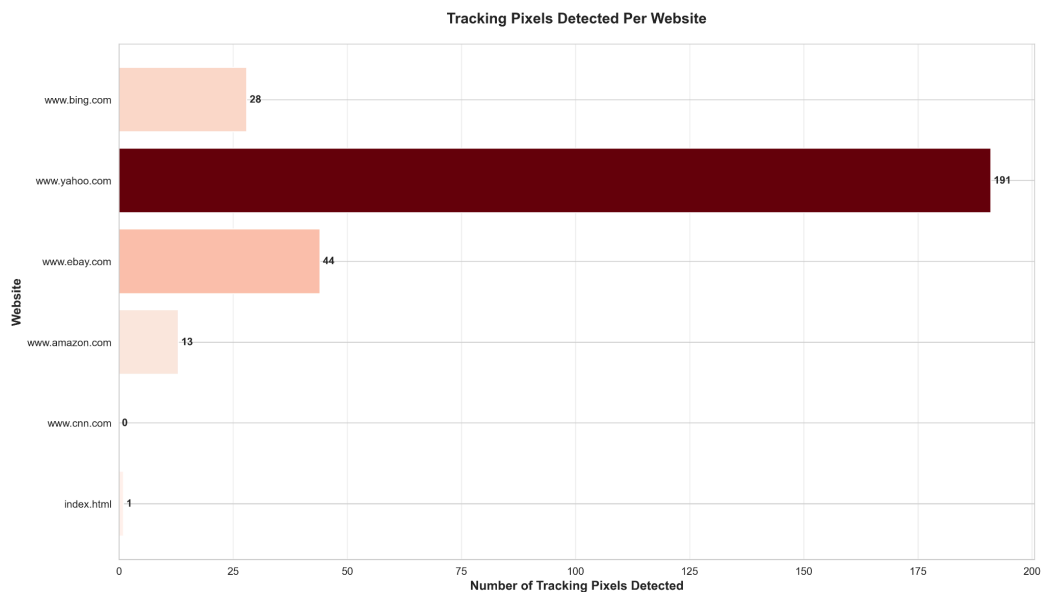


Figure 3



References

- Piixel leaks: Passive identification of personally identifiable. (n.d.).
<https://www3.cs.stonybrook.edu/~mikepo/papers/piixel.ccs25.pdf>

Appendix A Slides pdf

 Browser Sec Final Presentation

Appendix B (optional) github repo/demo video link

GitHub Repository: [monoPix-scout](#)